Arquitectura y Desarrollo de un Agente Conversacional Financiero para Microcomerciantes: Estrategias de Ingeniería para el Reto Deuna

Introducción y Contextualización del Desafío Corporativo

La democratización de la inteligencia de negocios representa uno de los retos tecnológicos y sociológicos más significativos en el sector financiero contemporáneo, particularmente en mercados emergentes y economías en desarrollo como la de Ecuador. Los microcomerciantes, a menudo propietarios de pequeñas "tiendas de barrio", generan una cantidad inmensa de datos transaccionales a través de la adopción de billeteras digitales y aplicaciones de cobro como Deuna.[1] Estos actores económicos constituyen una parte fundamental del tejido comercial y social del país, enfrentando desafíos diarios que van desde la inseguridad ciudadana y la crisis energética provocada por apagones, hasta la creciente competencia de cadenas de tiendas de descuento o "hard discount".[2] A pesar de poseer un volumen considerable de información sobre el rendimiento de sus propios negocios, la brecha de alfabetización financiera y tecnológica convierte a los reportes de datos tradicionales en herramientas ineficaces y cognitivamente abrumadoras.[1] Los cuadros de mando (dashboards) interactivos convencionales, los gráficos complejos de inteligencia de negocios y los reportes estáticos en formato PDF asumen un nivel de familiaridad analítica que el usuario promedio de este segmento simplemente no posee.[1]

Como consecuencia directa de esta fricción en la interfaz humano-computadora, las decisiones comerciales que son críticas para la supervivencia y el crecimiento del negocio se toman basándose en la intuición en lugar de apoyarse en la evidencia empírica extraída de los datos transaccionales.[1] El desarrollo de un asistente conversacional capaz de traducir datos estructurados a un lenguaje natural, empático y accionable promete transformar radicalmente esta dinámica, posicionando a la inteligencia artificial no como un simple motor de consulta de bases de datos, sino como un asesor financiero de confianza.[1]

El presente informe técnico detalla una arquitectura exhaustiva para construir dicho sistema, enmarcada en las restricciones y objetivos del reto corporativo "Mi Contador de Bolsillo". El entorno de desarrollo impone condiciones operativas estrictas: el sistema debe operar exclusivamente sobre un conjunto de datos sintético provisto por la organización, el cual contiene el historial transaccional de doce meses de tres comercios, totalizando aproximadamente dos mil transacciones por comercio, junto con un catálogo simulado de

clientes con identificadores anónimos y frecuencias de visita.[1] El agente conversacional resultante debe ser capaz de responder a consultas complejas de negocio en lenguaje natural, garantizando que el tiempo de respuesta por cada consulta no exceda los cinco segundos en el entorno de demostración.[1] Asimismo, se exige una precisión superior al ochenta por ciento en las respuestas evaluadas, la capacidad de incorporar visualizaciones simples dentro del flujo del chat, y el manejo de al menos un flujo de interacción proactiva donde el asistente alerte al comerciante sobre tendencias relevantes sin necesidad de una solicitud explícita.[1]

Para lograr estos objetivos, es imperativo descartar arquitecturas ingenuas y adoptar un paradigma de Análisis de Datos Agéntico (Agentic Data Analysis), aprovechando infraestructuras de computación de alto rendimiento (HPC) y modelos de lenguaje de código abierto finamente sintonizados para la generación de código y consultas estructuradas.

El Paradigma de Recuperación de Datos: Descarte de RAG y Adopción de Text-to-SQL

La intuición técnica inicial de evitar la arquitectura de Generación Aumentada por Recuperación (RAG, por sus siglas en inglés) para la resolución de este caso de uso específico es fundamentalmente correcta y está respaldada por la literatura reciente sobre agentes de datos. Las bases de datos vectoriales y los modelos de incrustación (embeddings) están intrínsecamente optimizados para la recuperación de similitud semántica en corpus de texto no estructurado, como manuales corporativos, artículos de investigación o normativas legales.[3] Sin embargo, estos enfoques probabilísticos fracasan estrepitosamente al intentar realizar operaciones analíticas precisas sobre datos estructurados tabulares, tales como operaciones aritméticas exactas, agregaciones temporales, conteos condicionales o uniones de tablas basadas en claves foráneas.[4] Si un comerciante pregunta "¿Cuánto vendí la semana pasada en comparación con el mes anterior?", un sistema RAG tradicional recuperaría los fragmentos de texto de las transacciones individuales que semánticamente se asemejen a los términos de la consulta, pero sería incapaz de ejecutar el cálculo matemático requerido para proporcionar una respuesta determinista.

La alternativa científica y arquitectónicamente rigurosa es la transición hacia flujos de trabajo de traducción de texto a lenguaje de consulta estructurado (Text-to-SQL) respaldados por modelos de lenguaje grande (LLMs) de razonamiento avanzado.[5] No obstante, el procesamiento de datos estructurados mediante LLMs ha estado históricamente dominado por promesas académicas que rara vez se materializan con la misma eficacia en entornos empresariales de producción.[6] Este fenómeno es conocido en la industria como el "abismo de rendimiento" (Performance Cliff) del Text-to-SQL.[6]

Métrica de Comparación	**Entornos Académicos (Spider 1.0)**	**Entornos Empresariales (Spider 2.0 / BIRD)**

Complejidad del Esquema	Bases de datos limpias, tablas pequeñas (3 a 10 tablas en total).	Cientos o miles de tablas, esquemas sucios y vistas parciales.
Nomenclatura de Columnas	Nombres legibles por humanos (ej. customer_name, invoice_total).	Nomenclatura corporativa críptica (ej. usr_trx_fl, txn_dt_01).
Ambigüedad Semántica	Consultas estructuralmente complejas pero sin ambigüedad en el significado.	Reglas de negocio ocultas (ej. definición exacta de "usuario activo").
Precisión de Ejecución Esperada	Rendimiento rutinario entre el 85% y el 92% de exactitud.	Caída dramática del rendimiento, a menudo entre el 6% y el 21%.

Las evaluaciones de referencia (benchmarks) tradicionales como Spider 1.0 miden la exactitud algorítmica en condiciones ideales.[6] Cuando los investigadores introdujeron métricas más representativas como Spider 2.0 y BIRD, que simulan la escala, el ruido y la complejidad de los almacenes de datos del mundo real, la precisión de ejecución de los modelos de vanguardia se desplomó.[6] Los errores lógicos silenciosos (Silent Logical Errors) se convirtieron en el modo de falla predominante: el LLM adivina incorrectamente la definición de un término de negocio y genera una consulta SQL que se ejecuta sin errores de sintaxis, pero que devuelve información numéricamente incorrecta, lo cual violaría directamente el requisito del ochenta por ciento de precisión establecido para el reto.[1]

Diseño e Implementación de la Capa Semántica

Para el conjunto de datos transaccional sintético proporcionado, que abarca un historial de doce meses y un catálogo de clientes, el volumen de datos (aproximadamente seis mil transacciones en total) no representa un desafío de almacenamiento, pero la interpretación de las relaciones sí constituye un desafío de razonamiento para el LLM.[1] La estrategia de mitigación más efectiva para alcanzar los niveles de confiabilidad académica en un entorno empresarial es la implementación de una capa semántica estricta y limitante.[6]

El objetivo principal de la capa semántica es evitar que el modelo de lenguaje deba "adivinar" la lógica de negocio subyacente.[6] En lugar de exponer el modelo a las tablas transaccionales en bruto con docenas de columnas de metadatos, se le deben presentar un número reducido de vistas (views) precalculadas y curadas, o integrarlo con herramientas de métricas gobernadas (similares a arquitecturas como dbt, Cube o Malloy).[6] Al limitar el alcance visual del LLM a, por ejemplo, cinco a diez vistas específicas (tales como Ventas_Diarias_Acumuladas, Frecuencia_Clientes, Productos_Mas_Vendidos), se reduce drásticamente la complejidad combinatoria que el modelo debe procesar.[6]

En este diseño, la capa semántica actúa como un diccionario determinista que mapea la intención en lenguaje natural a definiciones de métricas inmutables. Si la consulta involucra la palabra "clientes que no han vuelto", el motor semántico instruye al modelo explícitamente sobre el umbral de días inactivos que define a un cliente perdido según las reglas de negocio, eliminando la variabilidad inherente de las respuestas generadas por inteligencia artificial y asegurando que las cifras presentadas al microcomerciante sean auditables y matemáticamente consistentes.[1]

El Motor de Ejecución Analítica: Supremacía de DuckDB sobre Pandas

Una vez que el modelo de lenguaje ha generado exitosamente una consulta SQL a través de la capa semántica, el sistema requiere un motor de ejecución capaz de procesar la instrucción sobre el conjunto de datos y devolver los resultados estructurados al agente para su síntesis final. Históricamente, las bibliotecas de manipulación de marcos de datos (DataFrames) en Python, particularmente Pandas, han sido el estándar de la industria para el análisis en memoria. Sin embargo, para flujos de trabajo generados dinámicamente por inteligencia artificial, delegar la creación de código procedimental en Python a un LLM introduce una latencia significativa y una mayor probabilidad de fallos en tiempo de ejecución.[7]

El consenso en la ingeniería de datos moderna y el desarrollo de agentes LLM favorece contundentemente a DuckDB como el motor analítico local de facto para resolver este tipo de restricciones de baja latencia.[9]

Característica del Motor	**Pandas (Python)**	**PySpark**	**DuckDB**
Paradigma de Ejecución	Ansiosa (eager) y mayormente de un solo hilo	Computación distribuida, excelente para petabytes de	Procesamiento analítico en línea (OLAP), vectorizado y

	(single-threaded).	datos.	multiproceso.
Gestión de Memoria	Alta presión de memoria; los datos deben caber completamente en la RAM.	Requiere clústeres gestionados y configuración de la Máquina Virtual de Java (JVM).	Almacenamiento en columnas (columnar); ejecución eficiente en memoria con capacidad de desbordamiento a disco.
Interfaz de Consulta	API de Python orientada a objetos (requiere que el LLM escriba código Python).	Sintaxis de DataFrame y SQL dialectal de Spark.	SQL estándar nativo; los LLMs son expertos en escribir consultas directas.
Rendimiento Relativo	Más lento en agregaciones grupales complejas sobre miles de filas.	Sobrecarga de inicio alta (overhead) inaceptable para consultas de 5 segundos.	Extremadamente rápido en operaciones GROUP BY, SUM, AVG y escaneos de archivos.

DuckDB es un sistema de gestión de bases de datos relacionales en proceso, diseñado específicamente para soportar cargas de trabajo analíticas de alto rendimiento.[9] Su motor de ejecución vectorizado y pipelined procesa grandes lotes de datos simultáneamente, maximizando la eficiencia de la memoria caché de la unidad central de procesamiento (CPU).[8] Las evaluaciones de rendimiento comparativas (benchmarks) demuestran que, para consultas analíticas sobre conjuntos de datos con miles o millones de filas, DuckDB supera a Pandas, PySpark y SQLite por márgenes considerables en velocidad de procesamiento.[8] En pruebas de agregación filtrada y uniones de tablas cruzadas impulsadas por LLMs, los agentes basados en PandasAI (que generan código Python) consistentemente muestran tiempos de ejecución más lentos que los agentes LangChain respaldados por motores SQL directos.[13]

Más críticamente, los modelos fundacionales de lenguaje (especialmente aquellos entrenados en repositorios masivos de código fuente) poseen una profunda comprensión semántica y sintáctica del lenguaje SQL estándar.[10] Al acoplar un motor DuckDB con un agente de

enrutamiento SQL, el ciclo iterativo de inspección de esquema y redacción de consultas se vuelve altamente resistente a errores.[10] Además, DuckDB puede consultar directamente archivos en formato CSV o Parquet sin necesidad de un paso de importación o carga previa (no import step), eliminando el retardo de inicialización y asegurando que el tiempo de ejecución analítico consuma solo una fracción del presupuesto de cinco segundos, dejando el resto del tiempo disponible para la inferencia generativa del LLM.[10]

Las mejores prácticas para la implementación de agentes Text-to-SQL utilizando DuckDB dictan que se debe instruir explícitamente al modelo mediante el prompt del sistema para que utilice el dialecto específico de DuckDB, evitando que el agente asuma funciones exclusivas de PostgreSQL o MySQL que causarían errores de ejecución.[10] Asimismo, se recomienda configurar la reflexión perezosa de tablas (lazy table reflection) en los conectores de bases de datos para evitar que el agente consuma tiempo valioso inspeccionando catálogos de tablas completos cuando solo requiere un subconjunto de los datos sintéticos.[10]

Orquestación del Agente Cognitivo: Arquitecturas Orientadas a Grafos

La mera capacidad de traducir texto a consultas SQL y ejecutarlas en milisegundos no es suficiente para construir un asistente conversacional útil y resiliente. El microcomerciante ecuatoriano no interactuará mediante instrucciones rígidas estructuradas lógicamente, sino a través de preguntas vagas, coloquiales y contextualmente dependientes. Esto requiere una capa de orquestación agéntica superior que pueda manejar la ambigüedad, planificar pasos de ejecución y recuperarse de errores de manera autónoma.

El ecosistema actual de desarrollo de agentes de inteligencia artificial está liderado por marcos de trabajo (frameworks) como LangChain (específicamente su módulo LangGraph), CrewAI, AutoGen y BeeAI.[14] Para la analítica de datos estructurados en un entorno de producción que exige alta velocidad de respuesta y un control de estados riguroso, las filosofías arquitectónicas difieren fundamentalmente.

CrewAI centra su diseño en la colaboración basada en roles, orquestando "equipos" de agentes autónomos donde cada uno tiene un objetivo y una estrategia de comunicación definidos.[16] Por ejemplo, un sistema en CrewAI podría tener un agente "Analista de Datos" que extrae los números, y un agente "Estratega Comercial" que redacta la respuesta final. Si bien este enfoque es excepcional para la investigación profunda o la creación de contenido iterativo, la naturaleza conversacional de paso de mensajes de agente a agente introduce una sobrecarga de latencia significativa (a menudo decenas de segundos), lo cual imposibilita cumplir con la restricción de tiempo de respuesta de cinco segundos requerida por el reto corporativo.[1] Además, los sistemas multi-agente conversacionales son susceptibles a bucles infinitos de retroalimentación si los agentes no logran un consenso rápidamente.

La decisión arquitectónica óptima es emplear LangGraph. Este marco proporciona un motor de flujo de trabajo basado en grafos algebraicos, diseñado para la ejecución con preservación de

estado (stateful), permitiendo procesos adaptativos con ramificaciones y bucles explícitos.[14] A través de un diseño de grafos cíclicos dirigidos, los desarrolladores pueden construir un flujo de trabajo iterativo conocido en la industria como el bucle "Piensa-Comprueba-Corrige" (Think-Check-Fix loop).[10]

Cuando el LLM recibe una consulta del usuario, el grafo orquesta el tránsito de la información a través de los siguientes nodos lógicos predefinidos:

1. **Nodo de Clasificación y Enrutamiento:** Evalúa semánticamente si la pregunta de entrada puede ser respondida utilizando exclusivamente los datos del esquema sintético provisto.[1] Si el usuario solicita información externa ajena al negocio (por ejemplo, resultados deportivos o predicciones meteorológicas), el agente rechaza la consulta inmediatamente, cumpliendo la restricción impuesta de no inventar datos ni consultar fuentes externas.[1]
2. **Nodo de Reflexión de Esquema e Inspección:** El agente invoca herramientas (tool calling) como sql_db_list_tables y sql_db_schema para comprender qué columnas, tipos de datos y filas de muestra tiene a su disposición en la base de datos DuckDB instanciada en memoria.[10]
3. **Nodo de Generación de SQL:** Utilizando el contexto de la capa semántica, el modelo redacta un borrador de la consulta SQL compatible con el motor analítico.[10]
4. **Nodo de Validación Estática (Revisor):** Antes de enviar la carga computacional al motor de bases de datos, el flujo transita por una herramienta de revisión asistida por LLM (como sql_db_query_checker). Este nodo evalúa la consulta en busca de problemas comunes: errores de entrecomillado, columnas de unión incorrectas, desajustes de tipos de datos o funciones inexistentes en el dialecto objetivo.[10]
5. **Nodo de Ejecución y Auto-recuperación (Self-Healing):** La consulta validada se ejecuta contra el conjunto de datos sintético. El diseño del grafo dicta que si la base de datos devuelve un error sintáctico o una excepción de ejecución (por ejemplo, una división por cero), la traza de la excepción no se muestra al usuario. En su lugar, el estado se redirige de vuelta al Nodo de Generación, permitiendo que el LLM lea el mensaje de error, comprenda su equivocación, repare la sintaxis de la consulta y la vuelva a ejecutar de forma autónoma.[10] Este ciclo interno ocurre en milisegundos y es el mecanismo central que eleva la precisión de ejecución de las herramientas Text-to-SQL por encima de los enfoques de una sola oportunidad (one-shot prompting).[10]
6. **Nodo de Síntesis del Lenguaje:** Finalmente, el resultado tabular crudo se ingesta nuevamente en el LLM junto con la pregunta original, instruyendo al modelo a articular una respuesta en lenguaje natural empático, accionable y comprensible para un comerciante sin formación financiera.[1]

Esta tubería (pipeline) dirigida elimina el sobrecoste de planificación no determinista que sufren los frameworks menos estructurados, asegurando velocidad algorítmica y control total sobre las trayectorias de fallo.[10]

Selección de Modelos Fundacionales Abiertos para Razonamiento Estructurado

El rendimiento cognitivo del agente LangGraph depende inexorablemente del Modelo de Lenguaje Grande (LLM) subyacente responsable del enrutamiento de herramientas y la generación de código. El acceso a la infraestructura de Computación de Alto Rendimiento (HPC) a través de la Red Nacional de Investigación y Educación del Ecuador (CEDIA) otorga una ventaja asimétrica masiva para el equipo participante, permitiendo el despliegue local y privado de modelos fundacionales de código abierto (open-source) con un volumen de parámetros de grado empresarial que rivalizan directamente con las capacidades de sistemas propietarios comerciales como GPT-4o o Gemini 1.5 Pro.[1]

Para garantizar la precisión en el dominio específico del reto, la selección del modelo debe basarse en evaluaciones de referencia centradas en la generación de SQL analítico. Los resultados del exhaustivo *LLM SQL Generation Benchmark* ejecutado por plataformas analíticas como Tinybird, que miden la exactitud, eficiencia y latencia de consultas sobre bases de datos con cientos de millones de registros, revelan disparidades de rendimiento significativas entre arquitecturas aparentemente similares.[5]

Modelo de Lenguaje Evaluado	**Tamaño de Parámetros**	**Precisión en Generación SQL (Tasa de Éxito)**	**Clasificación Relativa en Código Abierto**
Llama 3.1 405B	405 Mil Millones	76%	Top Contendiente, rivaliza con modelos propietarios élite.
Llama 3.3 70B Instruct	70 Mil Millones	74%	Rendimiento excepcional con una fracción de los requisitos de memoria.
Codestral 22B	22 Mil Millones	70%	Altamente eficiente y especializado, excelente para entornos restringidos.

Llama 3.1 70B	70 Mil Millones	60%	Significativamente superado por las iteraciones más recientes (3.3).
DeepSeek Coder V2	236 Mil Millones (MoE)	Variable según benchmark	Estado del arte en código puro, pero requiere ajuste para análisis de negocio general.

Considerando la intersección entre el rendimiento absoluto de razonamiento lógico, los requerimientos de memoria de video (VRAM) y las métricas de latencia de inferencia, la familia de modelos Llama 3.3 de 70 mil millones de parámetros emerge como la selección técnica óptima.[5] Aunque Llama 3.1 405B ostenta el récord absoluto de precisión en código abierto, sus requerimientos de hardware masivos (necesitando múltiples nodos de GPU interconectados) complican el cumplimiento del estricto presupuesto de latencia de cinco segundos debido a la sobrecarga de paralelismo de red.[5]

Modelos especializados como DeepSeek Coder V2 (arquitectura de mezcla de expertos o MoE) exhiben un desempeño extraordinario en pruebas de finalización de código tradicional, pero su capacidad de razonamiento general para mantener una conversación empática con un usuario no técnico sobre la salud financiera de su negocio a menudo resulta ser menos robusta que la de modelos sintonizados específicamente para instrucciones conversacionales (Instruct-tuned) como los de Meta AI.[20] La serie Qwen 2.5 de 72 mil millones de parámetros también representa una alternativa viable y potente, particularmente por sus extraordinarias capacidades multilingües, lo cual asegura una fluidez absoluta en español.[20] Sin embargo, la amplia adopción comunitaria y la disponibilidad inmediata de perfiles de cuantización agresiva para Llama 3.3 inclinan la balanza hacia este último.

Infraestructura y Servicio de Inferencia en el Entorno HPC de CEDIA

La selección del modelo fundacional carece de utilidad práctica si el mecanismo de servicio (serving engine) introduce cuellos de botella que violan los requerimientos de latencia del negocio.[1] Para desplegar un modelo de setenta mil millones de parámetros y lograr que procese una consulta de usuario, transite por los múltiples nodos de razonamiento del grafo de LangGraph, genere la consulta SQL, y sintetice una respuesta natural en menos de cinco

segundos en total, la optimización extrema del servidor de inferencia es un requisito ineludible.

Limitaciones de Hardware y Cuantización de Modelos

Un modelo de 70 mil millones de parámetros cargado en la precisión de punto flotante estándar de dieciséis bits (FP16 o BF16) requiere aproximadamente 140 Gigabytes (GB) de memoria de video (VRAM) únicamente para almacenar los pesos estáticos del modelo, sin considerar el espacio dinámico necesario para el contexto de ejecución y la caché de atención.[23] Los servidores de alto rendimiento, como aquellos equipados con unidades de procesamiento gráfico (GPUs) NVIDIA A100, típicamente disponen de 40 GB u 80 GB de VRAM por tarjeta.[25] Consecuentemente, el modelo puro no puede instanciarse físicamente en una sola GPU A100, obligando al uso de partición de tensores (tensor parallelism) a través de múltiples tarjetas, lo cual incrementa exponencialmente los costos de interconexión y la complejidad del despliegue.[23]

La solución imperativa para este entorno es la aplicación de técnicas de cuantización post-entrenamiento (PTQ) de vanguardia, tales como GPTQ, AWQ, o la adopción emergente de tipos de datos nativos como FP8.[26] La cuantización comprime las representaciones matemáticas de las redes neuronales, reduciendo la precisión de los pesos de dieciséis bits a ocho o cuatro bits (enteros o flotantes).[28]

Cuantizar el modelo Llama 3.3 70B de FP16 a INT4 disminuye su huella de memoria estática en un 75%, reduciendo el requerimiento de almacenamiento de 140 GB a aproximadamente 35 GB.[23] Esta transformación volumétrica masiva permite que el modelo masivo resida holgadamente dentro de los límites de memoria de una única GPU NVIDIA A100 de 80 GB, dejando amplio espacio libre para la caché de contexto, lo cual es vital para procesar entradas largas.[23] Crucialmente, la literatura de investigación y los puntos de referencia técnicos demuestran que las metodologías de cuantización modernas infligen un daño estadísticamente insignificante a las capacidades de razonamiento del modelo; las evaluaciones en formatos cuantizados de 8 bits (w8a8) para Llama 3 reportan métricas de recuperación algorítmica superiores al 99.6% en pruebas estandarizadas de capacidad analítica como MMLU y GSM-8K frente a sus contrapartes de precisión completa.[29]

El Motor de Inferencia: vLLM vs Alternativas

En cuanto al software de servicio para la infraestructura CEDIA, la industria ofrece tres paradigmas dominantes: bibliotecas directas de Hugging Face (Transformers), motores enfocados en portabilidad y eficiencia de recursos en borde (como llama.cpp), y plataformas diseñadas expresamente para el rendimiento y la concurrencia a escala corporativa (como vLLM y TensorRT-LLM).[26]

La adopción de la biblioteca **vLLM** es la decisión arquitectónica obligatoria para superar el reto. Mientras que llama.cpp brilla por su diseño centrado en la CPU, su falta de dependencias externas y su soporte para el formato GGUF permitiendo el despliegue en computadoras portátiles o dispositivos perimetrales, su modelo interno de encolamiento y generación resulta

en una latencia entre tokens (Inter-Token Latency, ITL) constante pero subóptima para aplicaciones que demandan interactividad inmediata.[28] En contraste, vLLM está fundamentado en el innovador mecanismo de gestión de memoria *PagedAttention*.[26] Inspirado en la memoria virtual y la paginación de los sistemas operativos clásicos, PagedAttention gestiona de forma inteligente la caché de Clave-Valor (KV Cache) fragmentándola en bloques no contiguos en la memoria de la GPU.[26]

Este diseño arquitectónico casi erradica la fragmentación interna de la memoria, permitiendo al servidor agrupar de manera dinámica las peticiones asíncronas entrantes (continuous batching) con una eficiencia sin precedentes.[26] En configuraciones operativas, empresas tecnológicas reportan reducciones del 73% en los costos de inferencia tras migrar a vLLM, sosteniendo rendimientos masivos de cientos de tokens por segundo con latencias de percentil 99 (P99) excepcionalmente bajas.[26] Para el escenario de validación del jurado del hackathon, caracterizado por ráfagas de solicitudes concurrentes de baja latencia o interacciones de un solo usuario exigiendo respuestas inmediatas, vLLM sobre hardware HPC garantiza un Tiempo hasta el Primer Token (Time to First Token, TTFT) casi instantáneo, permitiendo que el LLM emita su respuesta a velocidades que superan los 25 a 40 tokens por segundo en modelos pesados, absorbiendo con holgura el margen de tiempo necesario para que el agente en LangGraph procese las llamadas iterativas a la base de datos DuckDB.[1]

Estrategias de Despliegue en Entornos SLURM

El despliegue de esta pila tecnológica en ecosistemas de supercomputación académica como el HPC de CEDIA introduce desafíos específicos relacionados con los sistemas de gestión de colas de trabajos, primariamente SLURM (Simple Linux Utility for Resource Management). En estos entornos multitenant, los investigadores y desarrolladores generalmente operan sin privilegios de administrador (root access) sobre los nodos de cálculo físico, lo que exige el uso riguroso de soluciones de contenedorización orientadas a HPC, tales como Singularity o Apptainer, en lugar de Docker tradicional.[34]

Un antipatrón arquitectónico sumamente común al instanciar inferencia distribuida o de alto rendimiento en clústeres SLURM ocurre al asignar recursos mediante comandos de trabajo programados (job arrays) o ejecuciones secuenciales con srun. Frecuentemente, los desarrolladores se encuentran con excepciones catastróficas del sistema operativo que dictan srun: error: unable to initialize step launch listening socket: Address already in use (Dirección de red ya en uso).[36] Este error paralizante es el resultado de un conflicto de contención de recursos a bajo nivel: los procesos subyacentes de comunicación paralela e interfaz de paso de mensajes gestionados por SLURM (como PMIx o MPI) colisionan con las bibliotecas de comunicación asíncrona distribuidas nativas de vLLM (como los backends de Ray o NCCL de NVIDIA) al intentar enlazarse simultáneamente a los mismos sockets o puertos lógicos de red dentro del hardware arrendado.[36]

Para garantizar un inicio inmaculado y confiable del servidor de inteligencia artificial dentro de los nodos HPC administrados por CEDIA, se deben emplear tácticas rigurosas de

configuración de entorno directamente inyectadas en los scripts de procesamiento por lotes sbatch:

1. **Aislamiento Dinámico de Puertos:** Es absolutamente crítico forzar al demonio de SLURM a seleccionar puertos dinámicos aleatorios para su infraestructura de comunicación interna de pasos entre nodos, evitando los puertos estáticos estándar. Esto se logra declarando explícitamente la variable de entorno export SLURM_STEP_RESV_PORTS=1 antes de la ejecución del contenedor.[36]
2. **Mitigación de la Contención de Sockets PMIx:** Se debe instruir a las bibliotecas subyacentes del protocolo PMIx para que alteren su comportamiento de resolución de red e impidan el conflicto de sockets compartidos. La inserción de los comandos export PMIX_MCA_gds=hash y export SLURM_OVERLAP=1 mitiga gran parte de los colapsos de interfaz.[36]
3. **Gestión Centralizada de Recursos mediante Clústeres Ray:** Si la complejidad del modelo (por ejemplo, si se opta por no cuantizar el modelo de 70B) requiere la implementación de Paralelismo de Tensores (Tensor Parallelism) distribuyendo las capas de atención a través de múltiples nodos físicos o servidores distintos en CEDIA, la documentación técnica y las mejores prácticas respaldadas por la comunidad de vLLM prohíben explícitamente iniciar múltiples instancias separadas de vLLM en trabajos de SLURM independientes.[37] El paradigma arquitectónico correcto consiste en solicitar una única asignación consolidada de todos los recursos (Nodos y GPUs) en un solo trabajo SLURM, proceder a instanciar de inmediato un clúster de procesamiento distribuido Ray (iniciando un nodo cabeza (head node) y uniendo los nodos trabajadores a este), y finalmente delegar el lanzamiento del motor vLLM exclusivamente como una tarea administrada dentro del plano de control de Ray.[35] Este enfoque garantiza que el planificador de Ray monopolice eficientemente el control de la memoria de la GPU y la topología de la red, evitando que los procesos huérfanos compitan destructivamente por los aceleradores de hardware subyacentes.[37]

Una vez estabilizado el entorno de ejecución, el servidor compatible con el protocolo OpenAI de vLLM expondrá una Interfaz de Programación de Aplicaciones (API) RESTful ultrarrápida, operando de forma segura dentro de la intranet del clúster académico, lista para ser consumida e iterada agresivamente por la interfaz del asistente conversacional desarrollada por el equipo del hackathon.[26]

Diseño de la Interfaz de Usuario y Visualización Multimodal Integrada

El pliego de condiciones técnicas del reto establece imperativamente la necesidad de desarrollar una interfaz de usuario mínima pero completamente funcional e interactiva, ejecutable como un chat web o una aplicación móvil.[1] Además, como un objetivo secundario crítico para destacar ante el jurado, se exige la incorporación dinámica de representaciones visuales clarificadoras, tales como gráficos de barras, gráficos circulares o tablas de datos

consolidadas, inyectadas directamente en el flujo de la conversación natural.[1] La premisa de negocio subyacente es clara: un comerciante sin formación financiera procesa la abstracción de tendencias comerciales con mucha mayor eficacia si las descripciones textuales van acompañadas de referencias visuales ancladas contextualmente.[1]

Para el desarrollo rápido y estructurado de aplicaciones web analíticas orientadas a inteligencia artificial en el ecosistema de Python, la biblioteca Streamlit ha ocupado históricamente la posición predeterminada en la comunidad.[39] Sin embargo, el paradigma de diseño central de Streamlit presenta limitaciones severas para casos de uso conversacionales complejos. Streamlit ejecuta la totalidad del código fuente (eager execution) de forma descendente tras cada interacción del usuario en la interfaz.[9] Esta recarga constante de la página complica extraordinariamente la persistencia en memoria del estado de la conversación, el historial del chat, y las conexiones activas a los motores analíticos de bases de datos, requiriendo implementaciones frágiles de almacenamiento de sesión en caché que añaden latencia y son propensas a comportamientos impredecibles.[40]

La decisión técnica óptima para garantizar un rendimiento fluido y escalable es el abandono de Streamlit en favor de **Chainlit**.[39] Chainlit es un marco de trabajo de código abierto de nueva generación diseñado específicamente, desde sus fundamentos, para la construcción y el despliegue de interfaces de usuario conversacionales (UIs) listas para producción, enfocadas en agentes y ecosistemas impulsados por modelos de lenguaje grande.[39] Su arquitectura nativa está profundamente alineada con las necesidades del proyecto: proporciona integraciones profundas y sin fricción con el ecosistema de LangChain, facilita el almacenamiento persistente de las sesiones de los usuarios, permite la visualización en tiempo real del razonamiento del agente (los pasos lógicos del flujo de LangGraph) para propósitos de depuración, y opera bajo un modelo de programación asíncrono orientado a eventos, desencadenando la lógica de negocio a través de decoradores estructurados como @cl.on_message en lugar de recargar archivos de script indiscriminadamente.[40]

Implementación de Gráficos Dinámicos en Línea (In-Line Visualizations)

Para que el asistente conversacional materialice su rol de "Contador de Bolsillo" integral, las respuestas generadas por el agente LLM deben trascender el texto plano y adoptar capacidades multimodales cuando el comerciante interroga al sistema sobre métricas comparativas temporales, tales como "¿Cómo se compara la venta del día de la madre con mis ingresos promedios del mes?" o "¿Cuál es la distribución de mis productos más rentables?".[1]

Aprovechando las potentes capacidades de enrutamiento y uso de herramientas (Tool Calling) del agente LangGraph configurado previamente, el sistema puede ser programado para evaluar de forma autónoma si los resultados numéricos extraídos del motor de bases de datos DuckDB se benefician de una abstracción gráfica. En caso afirmativo, la lógica del servidor de Python procede a renderizar programáticamente un objeto visual estético utilizando la popular biblioteca de trazado gráfico Plotly, generando representaciones dinámicas como diagramas

de dispersión, gráficos circulares o curvas de tendencia.[44]

El marco de trabajo Chainlit posee compatibilidad algorítmica nativa con objetos generados por Plotly a través de la instanciación de la clase especializada de renderizado cl.Plotly.[44] Esta arquitectura permite encapsular el objeto visual dentro de los metadatos del mensaje de respuesta. Al construir y emitir el evento de chat hacia el cliente web mediante estructuras de código simples e intuitivas—como `await cl.Message(content="Aquí está el desglose detallado de sus ventas semanales...", elements=[cl.Plotly(name="grafico_ventas", figure=fig, display="inline")]).send()`—la interfaz frontal renderiza un gráfico de alta fidelidad completamente interactivo incrustado en la burbuja de la conversación.[44]

Esta integración fluida permite que el microcomerciante final aísle variables de datos, realice operaciones de acercamiento (zoom) espacial sobre periodos de tiempo específicos e inspeccione tendencias o estacionalidades detalladas simplemente posando el cursor sobre los elementos del gráfico, sin la desorientación cognitiva que supone abandonar la ventana del chat para acceder a un panel externo de analítica de inteligencia de negocios.[44] Esta fusión armónica de texto generado algorítmicamente y representación gráfica de datos exactos cumple de manera rotunda, elegante y eficiente el requerimiento de incorporación visual solicitado por los jueces del hackathon.[1]

Inteligencia Ambiental: Implementación de Alertas Proactivas y Ejecución Asíncrona

El criterio diferenciador de mayor sofisticación técnica exigido por el jurado de Deuna para la evaluación de los proyectos es la implementación demostrable de al menos un flujo de interacción asíncrono y proactivo, donde el asistente cognitivo asuma la iniciativa de alertar al comerciante sobre una tendencia analítica relevante o un riesgo incipiente en su ecosistema de negocio, sin necesidad de que este formule una pregunta previa.[1] En la literatura académica y la ingeniería de arquitectura de inteligencia artificial a escala empresarial, este requisito representa el salto evolutivo crítico de un Agente Conversacional (Conversational AI), que aguarda pasivamente instrucciones y reacciona de manera aislada al estímulo del usuario, hacia la concepción de un Agente Ambiental (Ambient Agent) o Agente Proactivo (Proactive Agent) plenamente autónomo.[47]

Un agente verdaderamente proactivo opera como un centinela digital que vigila incansablemente las métricas transaccionales del negocio subyacente. No depende de los "prompts" de entrada del usuario en la interfaz del chat; su ejecución iterativa es desencadenada dinámicamente (triggered) por condiciones programadas: la llegada e ingesta de nuevos lotes de datos transaccionales de cobro, la activación de eventos internos específicos en la arquitectura de la base de datos, o la detección algorítmica de anomalías estadísticas severas.[47] Modelos estadísticos o herramientas configuradas en la inteligencia pueden detectar patrones de compras inusuales que sugieran transacciones anómalas, identificar una concentración de pagos rechazados, o simplemente percibir una desviación

estándar significativa en las series temporales de ingresos diarios, como una caída abrupta del rendimiento comercial durante una tarde de día laborable en comparación con la mediana estadística del último trimestre.[48]

Orquestación Asíncrona en la Capa de Interfaz y Lógica de Segundo Plano

A nivel de arquitectura de software y gestión de procesos del sistema operativo, integrar un comportamiento proactivo de emisión de alertas en un sistema de mensajería tradicionalmente síncrono presenta formidables desafíos técnicos de bloqueo de los hilos de ejecución de la interfaz gráfica. Si la rutina de evaluación analítica requiere tiempo computacional, la interfaz web del usuario podría congelarse o negarse a recibir nuevos mensajes hasta que el ciclo de monitoreo concluya.[50]

La ventaja insuperable de haber seleccionado Chainlit como el entorno subyacente radica en que el ecosistema del framework está diseñado para la programación asíncrona por defecto (async by default), construyéndose orgánicamente sobre las capacidades nativas de la biblioteca de operaciones asíncronas de E/S concurrente (asyncio) del lenguaje Python.[42] Esta arquitectura fundamental de no bloqueo (non-blocking) permite a los ingenieros de software construir aplicaciones donde decenas o cientos de usuarios mantengan sesiones activas simultáneamente en un único servidor de front-end, mientras que el procesador subyacente gestiona elegantemente operaciones de alto costo computacional o esperas de red en segundo plano sin interrumpir la fluidez de la interfaz.[51]

La instrumentación técnica de la alerta proactiva solicitada para la demostración del hackathon exige una canalización de mensajería precisa mediante tareas concurrentes. La mecánica del flujo se estructura en las siguientes etapas operativas clave:

1. **Detección y Monitoreo en Segundo Plano:** Al instanciarse la sesión de la aplicación web, o como un microservicio independiente operando perpetuamente, se lanza una rutina o corrutina secundaria asíncrona. Esta tarea paralela actúa como el monitor continuo del almacén de datos DuckDB. Mediante constructos temporales simples como bucles iterativos regulados por la instrucción suspensiva `await asyncio.sleep(intervalo_segundos)`, tareas administradas con la API `chainlit.run_task`, o la simulación de consumo de una cola de eventos webhooks que imitan la entrada de un nuevo bloque de datos en el entorno sintético de Deuna, el proceso evalúa silenciosamente subconjuntos del historial transaccional sin perturbar el ciclo vital principal de interacción humana.[51]
2. **Evaluación de Condiciones Lógicas Estrictas:** A intervalos regulares, el proceso asíncrono formula y transmite una consulta analítica ligera pero altamente optimizada a la capa semántica de DuckDB. El objetivo de la consulta es comparar las métricas móviles. Por ejemplo, el agente evalúa una regla de negocio: si los ingresos totales consolidados del día actual exhiben una regresión superior al veinte por ciento respecto a la mediana móvil consolidada de los martes de los tres meses anteriores, la condición paramétrica

de la alerta proactiva se satisface algorítmicamente y el evento se dispara internamente.[49]

3. **Generación y Síntesis Empática de la Alerta:** El evento disparador encapsula los datos crudos de la anomalía en un paquete JSON y los envía enrutados a una rama separada del grafo LangGraph. El agente LLM (Llama 3.3 70B en el nodo de Cedar) recibe la instrucción de formular un mensaje de interrupción altamente empático y contextualizado, explicando la alerta en términos comprensibles ("*Hola, he estado revisando los números en la caja y he notado que la venta de víveres disminuyó considerablemente esta tarde en comparación con lo que habitualmente logras. ¿Te gustaría que analicemos a fondo qué productos están rotando menos para armar una promoción especial de fin de día?*").[1]
4. **Inyección Asíncrona y Notificación Push In-Session:** La barrera técnica final es empujar físicamente el mensaje a la interfaz del comerciante. Aprovechando los ganchos del ciclo de vida asíncronos en Chainlit, el hilo operativo secundario puede forzar el envío del mensaje push generado directamente al objeto de la sesión conversacional activa. Esto se logra ejecutando imperativamente funciones de envío como `await cl.Message(content="Alerta proactiva:...").send()` explícitamente fuera del decorador regular y convencional `@cl.on_message`, el cual está habitualmente reservado de manera exclusiva para escuchar y procesar las interacciones de entrada del usuario humano.[51]

Esta inyección programada y deliberada de la interrupción informativa dentro de un hilo de conversación que no la solicitó atrae la atención inmediata y enfocada del microcomerciante directamente sobre la pantalla de su terminal web o dispositivo móvil.[55]

Esta reingeniería de la relación entre el dato inerte y el usuario final transforma un repositorio relacional pasivo de información corporativa en un participante alerta y activo de los intrincados procesos diarios del local minorista.[47] Más importante aún, ataca directamente y mitiga el problema neurocognitivo fundamental y la hipótesis de negocio expuesta por el equipo corporativo de Deuna en las bases del reto: los propietarios de los negocios terminan operando y tomando decisiones críticas basadas en la pura intuición intuitiva y corazonadas operativas porque carecen del tiempo material, la paciencia o la energía analítica para sentarse a examinar proactivamente tableros abrumadores o abrir reportes financieros densos en PDF.[1]

Cuando la arquitectura de software asegura que la inteligencia de valor incalculable viaja sin fricción hacia la atención visual del tendero, precisamente en el momento algorítmico exacto de la ineficiencia empresarial o la necesidad operativa apremiante, las barreras originadas por la falta de alfabetización contable, financiera y tecnológica del segmento de usuarios pasan inmediatamente a un plano irrelevante.[1] Todas estas deficiencias estructurales son elegantemente absorbidas y neutralizadas por la extrema autonomía analítica del agente ambiental.

Localización Semántica, Construcción del Prompt de Sistema y Guardarraíles de Precisión

Superados los inmensos obstáculos de la ingeniería de la base de datos analítica vectorizada, la computación orquestada en clústeres de supercomputación y las arquitecturas de interfaz de usuario concurrentes y multimodales, el sistema se enfrenta a una barrera final, profundamente enraizada no en la tecnología computacional, sino en la psicología conductual y la antropología sociolingüística. El impedimento final para lograr la adopción y retención tecnológica en un segmento de mercado compuesto por propietarios de tiendas barriales no radica en los algoritmos predictivos o los microsegundos de latencia, sino en la empatía y la sintonización discursiva de la comunicación.

Un modelo de inteligencia artificial genérico sin ajustar o en su estado puro pre-entrenado (incluso uno tan avanzado como Llama 3 70B Instruct), por su propia naturaleza estadística de haber devorado corpus enteros de internet, se expresará por defecto utilizando un tono extremadamente académico, corporativo, seco, y muy probablemente adoptando un registro de léxico y modismos españoles orientados preferentemente hacia el público internacional estándar o peninsular ibérico.[57] Para que un "tendero" del territorio ecuatoriano deposite su confianza y perciba al asistente no como un robot ajeno a su realidad material, sino como un "asesor de confianza" empático, genuino y verdaderamente útil, el sistema en su totalidad debe comunicarse exactamente en la misma longitud de onda semántica, léxica y cultural del usuario.[1]

El diseño cuidadoso, riguroso y sistemático del mensaje rector general del comportamiento del asistente cognitivo (conocido técnicamente en la ingeniería de IA como el *System Prompt*) constituye la piedra angular y el plano maestro para alinear herméticamente los vastos conocimientos de la inteligencia artificial general con la singularidad del ecosistema sociolingüístico comercial específico del Ecuador.[58] Este mensaje fundacional debe estar exhaustiva y metodológicamente estructurado, rehusando el empleo de directrices vagas o difusas, y operando con rigor bajo patrones arquitectónicos de ingeniería de instrucciones iterativos (Prompt Engineering frameworks) avanzados y de alto nivel corporativo.[57]

El Prompt de Sistema como Controlador Categórico de la Semántica

En este contexto riguroso de hackathon, donde cada palabra cuenta hacia la percepción cualitativa del jurado calificador, un mensaje de sistema robustamente estructurado asume tres frentes de responsabilidad crítica para garantizar el éxito de la solución de software [59]:

1. **Implantación Rigurosa de Persona y Limitación del Tono Operativo:** Es imperativo programar algorítmicamente al LLM, en la capa más base de su sistema de atención lógica, para que actúe de manera inmutable como un asesor financiero amistoso, profundamente empático, paciente, y sin embargo, de un rigor y utilidad absolutos. Las directrices estrictas codificadas en las matrices de reglas del *prompt* principal deben prohibir de forma categórica e irrenunciable el uso de jerga financiera, jerga

administrativa corporativa y términos contables o anglicismos complejos de uso en capital de riesgo.[1] Términos técnicos como "Análisis EBITDA", "Tasa general de Churn rate de retención", "Porcentaje de Retorno de inversión puro", "Ratio de flujo de caja libre" o "Indicadores KPI analíticos" deben estar explícitamente prohibidos o listados en un bloque de exclusión negativa en las instrucciones maestras del agente, obligando coercitivamente al modelo a generar ciclos de procesamiento que traduzcan internamente estos conceptos teóricos a realidades materiales cotidianas y explicaciones simplificadas. Por ejemplo, "Análisis de ganancias operativas y flujo" debe reconvertirse a frases tangibles y directas como "El dinero contante y sonante que te quedó en el bolsillo al final del mes después de pagar todos tus gastos", o referirse a los "indicadores de rotación y pérdida de clientes (churn)" simplemente como "El número total de clientes y vecinos fieles que lastimosamente han dejado de visitarte este último trimestre".[1]

2. **Adaptación Dialectal Profunda y Comprensión del Entorno Andino:** Aunque las restricciones impuestas de antemano dictan que el agente robótico debe expresar las respuestas verbales y escritas en "español neutro, comprensible para un adulto" con el propósito de maximizar su accesibilidad transversal, el entendimiento lingüístico interno e implícito (la capa de ingesta y comprensión del modelo) de las consultas informales arrojadas por el comerciante sí debe abarcar la riquísima sociolingüística y los modismos idiosincráticos del bullicioso ecosistema comercial informal característico de las urbes del Ecuador. El LLM masivo desplegado debe ser inyectado contextualmente o suplementado agresivamente mediante técnicas de aprendizaje de pocos ejemplos (Few-Shot prompting incorporados en el encabezado del sistema) para lograr comprender algorítmicamente y sin la menor duda de latencia que ciertos términos folclóricos que el usuario ingresará en su chat son sinónimos técnicos inalterables de conceptos de base de datos comerciales precisos. Debe entender de inmediato que referencias al *tendero* se traducen como propietario o usuario principal del sistema; que el término profundamente arraigado de la *yapa* equivale a un valor o volumen añadido de forma gratuita en la finalización de una transacción para fidelizar al comprador habitual; o que el concepto recurrente de la contabilidad comunitaria del *fiado* equivale a la emisión de crédito informal descentralizado sin interés temporal y con registro rudimentario de deudas a corto plazo.[2] Si el sistema falla en comprender esta gramática y demanda constantes aclaraciones semánticas, destruirá instantáneamente la frágil ilusión cognitiva de asesoría comercial amigable de la que depende enteramente la usabilidad del prototipo.[1]
3. **Seguridad Arquitectónica Inflexible y Restricciones Absolutas de Datos (Anti-Hallucination Guardrails):** Dado que la integridad final de los resultados reportados en las demostraciones en vivo evalúa un criterio de éxito riguroso del ochenta por ciento de respuestas verificadas y exige, por encima de todo, una restricción absoluta y sin excepciones éticas sobre la invención, pre-cálculo falso o interpolación imaginaria de datos [1], el marco estructurado del prompt maestro debe establecer un bloque de reglas lógicas o heurísticas de ejecución deterministas inflexibles que actúen como un seguro perimetral de confianza operativa sobre el Agente conversacional:

- *Regla Maestra 1: Limitación de Dominio Restringido*. Si la temática subyacente a la consulta ingresada por el usuario comerciante no pertenece estrictamente a los vectores de dominio de los datos transaccionales, o no se relaciona lógicamente con la salida de las herramientas y métricas de funciones SQL expuestas al agente, este se encuentra bajo la directiva absoluta de declarar explícitamente y con cortesía su completa ignorancia sistémica, negándose tajantemente a consultar u ofrecer información exógena inventada, hipotética o generalizada de la web o de su peso de entrenamiento, dando así cumplimiento legal a la restricción fundamental de no consultar ni apoyarse jamás en fuentes lógicas externas.[1]
- *Regla Maestra 2: Dependencia Total del Contexto Retornado*. El razonamiento matemático y numérico del agente, y todas y cada una de sus afirmaciones con respecto a cifras monetarias, cantidades métricas y métricas de desempeño, deben apoyarse única y exclusivamente en el procesamiento analítico interno de las listas en formato JSON o los conjuntos de arreglos de formato tabular estricto que le han sido devueltos a la fuerza mediante su interacción programática indirecta con el motor de base de datos OLAP DuckDB, negándole de forma preventiva y radical cualquier intento ingenuo de resolver matemáticas aritméticas por cuenta de su propio generador de inferencia lógico y estadísticamente falible (impidiendo así a la red neuronal utilizar vectores estáticos probabilísticos para ejecutar cálculos de base contable que deben ser algorítmicos exactos).[1]
- *Regla Maestra 3: Tratamiento Lógico de Conjuntos Vacíos Excepcionales*. Si la ejecución sistemática de la consulta semántica convertida a dialecto SQL correcto hacia el dataset devuelve por cualquier medio analítico un conjunto de información vacío (vectores en blanco, resultados igual a null, un dataframe sin índice, o arrays de cero ocurrencias), la inteligencia artificial debe ser obligada a procesar este vacío y derivar lógicamente en su escritura y comunicación verbal final hacia la interfaz del usuario que la información técnica o transaccional explícitamente solicitada simplemente no existe dentro de la ventana de análisis presente del esquema sintético. O bien, que matemáticamente no se ha registrado un hito o se han realizado transacciones comerciales en el segmento estacional señalado. En ningún caso bajo este estado o árbol condicional se le tolerará al módulo predictivo intentar derivar proyecciones abstractas, adivinar conjeturas basadas en tendencias históricas interpoladas y no confirmadas, o formular proyecciones temporales sin evidencia fáctica.[1]

Al someter en cadena toda la capa final y abstracta de traducción lingüística asincrónica de la solución de software general a este conjunto de estrictas y blindadas restricciones programáticas semánticas, operacionales e inflexibles, la solución tecnológica global erradica algorítmicamente y elimina virtualmente el considerable riesgo sistémico latente que persigue a los chatbots modernos.[17] Es decir, extirpa desde su origen la posibilidad trágica de que una plataforma o inteligencia artificial concebida como asesor y herramienta constructiva, guiada ciegamente por fallas estadísticas en su comprensión, termine generando una cascada analítica irreal y falsa que induzca activamente a un microempresario sin mayores defesas

cognitivas a acometer decisiones y riesgos financieros comercialmente catastróficos para el futuro existencial de su modesto comercio, basándose de manera catastrófica en simples alucinaciones y espejismos provenientes del ruido estadístico profundo de la red generativa.

Síntesis Tecnológica Integral y Estrategia para la Demostración

El desarrollo y concepción holística del sistema conversacional "Mi Contador de Bolsillo" enfocado en micro-comercios y propuesto para abordar con ambición las demandas estipuladas en el Reto 2-IA de la corporación Deuna, trasciende con mucho la simple manipulación experimental de un modelo comercial basado puramente en la envoltura en torno a una sencilla Interfaz de Programación de Aplicaciones de inteligencia en la nube. La construcción exitosa de esta plataforma operativa demanda una intersección orquestada y asombrosamente precisa entre recursos pesados de infraestructura de procesamiento en centros de supercomputación, motores relacionales compactos de bases de datos altamente optimizados de ejecución rápida, lógicas avanzadas de enrutamiento agéntico y una profunda afinación heurística de modelos fundacionales lingüísticos masivos especializados.[1]

En retrospectiva sobre todo el andamiaje y la evaluación técnica exhaustiva explorada meticulosamente en este informe, la ruta ingenieril crítica orientada inexorablemente hacia la consecución de la absoluta viabilidad operativa, el cumplimiento estricto y total de las difíciles rúbricas de evaluación del jurado corporativo, y una ostensible ventaja competitiva de primer nivel frente a los enfoques ingenuos y convencionales de otros equipos inmersos en la arena de competencia del hackathon nacional *Interact2Hack*, debe obligatoriamente anclarse y sintetizarse de forma dogmática en el acatamiento indiscutible de las siguientes determinaciones arquitectónicas de grado industrial, que pasan a ser consideradas elementos de carácter vinculante e innegociable en el diseño constructivo:

En primer lugar, el abandono consciente, fundado y explícito de cualquier tentativa que busque construir un sistema que confíe ciegamente en canalizaciones estructuradas a través de la arquitectura probabilística de Generación Aumentada por Recuperación (el popularizado paradigma RAG).[3] Queda evidenciado más allá del plano teórico que las búsquedas cimentadas en la distancia o proximidad de representación vectorial de elementos en espacios multidimensionales no ofrecen ninguna clase de garantía matemática algorítmica ni exactitud material posible cuando el sustrato del problema aborda inexorablemente consultas de suma total o evaluación de cientos o miles de registros y filas numéricos agregados procedentes de tablas comerciales de finanzas y caja transaccional.[5] El núcleo del sistema adoptará imperativamente y sin reservas una canalización de inferencia profunda de lenguaje humano convertido o traducido lógicamente al lenguaje de consultas estructuradas (Paradigma de Análisis Agéntico de Text-to-SQL analítico). Esta canalización abstracta generada estará intrínseca e indisolublemente vinculada con, y delegada directamente a, un motor local veloz instanciado y montado puramente en la memoria RAM del sistema subyacente; el aclamado motor OLAP DuckDB, el cual posee una ventaja inigualable y es de facto irremplazable por su

excelencia sin paliativos ejecutando operaciones analíticas, sumatorias masivas temporales iterativas y lógicas condicionadas de bases vectorizadas masivas, superando radicalmente a herramientas de data frame procedimentales genéricas como los anticuados bloques de carga ansiosa (eager) de Pandas Python y otras utilidades que introducirían una latencia absolutamente inaceptable superando el tiempo vital límite exigido de respuesta en vivo.[9]

En segundo orden fundamental de importancia analítica, se encuentra el absoluto e infranqueable aislamiento semántico y programático del motor analítico expuesto hacia la entidad inteligente. La base de datos cruda y con terminología en bruto corporativa jamás será expuesta o filtrada directamente al modelo de lenguaje puro. Todo el historial de transacciones subyacente a la capa relacional se encapsulará defensivamente por debajo de un estrato semántico riguroso de abstracción lógica superior.[6] Bajo ninguna circunstancia o instrucción de recuperación de error se dejará o facultará al agente LLM la riesgosa y costosa tarea de intentar descifrar estadísticamente por su cuenta las correlaciones complejas entre los miles de eventos históricos; en sustitución y por imposición restrictiva, la inteligencia simplemente deberá ejecutar un mapeo iterativo y algorítmico, guiando la intención difusa verbalizada o inferida de las oraciones tipeadas y redactadas por el modesto tendero local directamente y en un puente fijo hacia un bloque precalculado e higienizado o una vista consolidada estática que ya ha sido gobernada, indexada, verificada y previamente validada matemáticamente por diseño del sistema de analítica y métrica de la información, mitigando y eludiendo inteligentemente a nivel topológico estructural de raíz el desastroso precipicio del abismo de precisión, que estadísticamente destruye cualquier aplicación inmadura de Text-to-SQL desplegada de forma cruda sobre repositorios complejos del mundo real empresarial.[6]

Como tercer punto, la arquitectura de control y flujo operativo lógico conversacional se fundamentará y basará estricta e inexcusablemente en la orquestación direccional a través del avanzado y robusto mecanismo de máquinas y grafos de estado cerrado iterativo representados por el framework LangGraph, desestimando de inmediato opciones superficiales centradas en roles meramente conversacionales que descontrolan radicalmente las variables de latencia general e introducen la insidiosa planificación ambigua multi-agente que satura y desestabiliza el tiempo por respuesta de la demostración en red.[14] A través de las rutas topológicas deterministas que dibujan el recorrido del grafo en un bucle perenne de generación de esquemas de interrogación, una posterior verificación asertiva contra bases dialécticas, la remisión del mandato ejecutivo subyacente a la base, y de ser interceptado cualquier tipo de excepción de compilación relacional o falta temporal de información cruzada, la activación de un ramal curativo de auto-corrección regenerativa asistida.[10] Esto garantizará matemáticamente como una función del flujo cerrado en bucle inquebrantable de repetición, que la tubería computacional terminal en su fase de entrega cumpla siempre y consistentemente y apruebe en cada intento de demostración la restricción dura de la evaluación del jurado de proveer una síntesis conversacional integral a partir de transacciones reales en el exigido marco y umbral crítico inferior a la barrera tope de cinco segundos en un terminal asíncrono y de usuario único.[1]

Como pilar tecnológico cuarto en la edificación operativa de esta magna tarea ingenieril,

resalta como prioridad principalísima y táctica insoslayable la capitalización masiva, exhaustiva y táctica de los inconmensurables recursos e instancias físicas de hardware provistos por las potentes asignaciones académicas y terminales dedicadas dentro del CEDIA HPC para el clúster. La ventaja asimétrica innegable se obtendrá materialmente empleando y hospedando de manera privada masivos y ultra avanzados modelos fundacionales en configuraciones pesadas de código abierto—con un énfasis prioritario y explícito en el impresionante bloque de conocimiento que porta inherentemente pre-entrenado el titánico **Llama 3.3 70B Instruct** u otro de arquitectura Mixture-of-Experts y peso superior—al cual se debe someter urgentemente y sin alternativas a una rutina agresiva e intrusiva de reducción volumétrica vía cuantización post-entrenamiento paramétrica de nivel w8a8 hasta límites de cuatro bits como los provistos a través de frameworks de afinado espacial AWQ o de reticulación escalar directa GPTQ.[5]

Esta drástica alteración posibilitará el calce paramétrico perfecto y asegurará el anclaje físico, volumétrico y total del espectro cognitivo necesario de todo el bloque del modelo de lenguaje comprimido pero intachablemente astuto en la arquitectura de memoria VRAM delimitada en las GPUs masivas NVIDIA A100 provistas de la infraestructura de hardware, propiciando que se maximize y despunte exponencialmente y hasta la exasperación matemática la enorme destreza resolutiva analítica computacional requerida imperiosamente a través de un servicio de puente de backend optimizado de vLLM o TensorRT con rutinas de gestión PagedAttention, mientras paralelamente mantiene e invalida al instante las penosas fricciones por latencia del primer token. Todo el proceso operando sobre el resguardo y escudo unificado que brinda la capa orquestadora multi-recurso e inferencia remota de contenedores paralelos en enrutadores dinámicos del sub-planificador clúster de interfaz general Ray corriendo por encima del programador rígido tradicional multi-usuario y secuencial que gobierna en los nodos paralelos las tareas operativas de las colas eslurm, evadiendo los letales conflictos ciegos por red bloqueada y el catastrófico asalto por puertos de la infraestructura compartida CEDIA.[26]

Finalmente, como la pieza que corona de manera incuestionable el esfuerzo programático final hacia la innovación y asombro visual e interactivo requerido y fuertemente calificado en la exposición por el consorcio de los jueces ejecutivos, subyace la imperativa asimilación técnica total de una interfaz reactiva puramente asíncrona a nivel de su concepción base y en cada nivel o plano paralelo del servicio. Se debe rehusar de manera inequívoca y absoluta la tentación cortoplacista de construir un front-end frágil basado en bibliotecas estáticas convencionales enfocadas tradicionalmente en rudimentarios cuadros de mando monolíticos que arrastran consigo las penurias y el agotamiento de recursos característicos de las rutinas completas de recarga pesada sincrónica imperativa en cada pequeña iteración visual superficial, descartando a Streamlit y transicionando en favor de plataformas modernas orientadas por eventos paralelos, tal como lo establece el canon del framework asíncrono **Chainlit**.[9]

La adopción de la versatilidad reactiva y persistencia profunda propia de Chainlit desatará y liberará toda la potencia contenida de la inyección nativa asíncrona de ricos componentes y

gráficas matemáticamente renderizadas de primerísima y muy alta fidelidad directamente procesadas en línea desde Plotly o constructos de Pyplot.[44] De modo trascendental e insustituible, será este robusto control e interacción el que finalmente habilite y haga realidad técnica posible y fehacientemente comprobable la inserción proactiva asombrosa, inesperada y mágica en el plano asíncrono del backend persistente, la creación innovadora de la tan elusiva, asíncrona y valiosa rutina secundaria silenciada que operará autónomamente detrás del telón invisible de la interacción, permitiendo que la creación y el modelo pasen en esencia de estar calificados y limitados por un abanico funcional genérico como un asistente inerte y rudimentario que aguarda pasivo a las directrices como las viejas barras de búsqueda planas, a transmutarse cognitiva y programáticamente e irradiar vida operativamente como si se tratara de un verdadero Analista Experto del comportamiento bursátil microeconómico y el primer genuino acompañante asíncrono empresarial o de la estirpe de vanguardista del "Agente Inteligente Ambiental y Proactivo" ininterrumpidamente atento, revolucionario y sin paralelos en este contexto local andino.[44]

Al alinear estas cinco piezas teóricas e infraestructurales, el sistema superará holgadamente las métricas del reto exigidas sin invención y con total velocidad exigible por la demostración final, constituyéndose además como un genuino prototipo arquitectónico vanguardista digno de retener y escalar industrialmente la esencia y el capital indispensable del sector comercial más extendido, vibrante, indispensable e integrador socioeconómicamente hablando del país entero.[1]

Obras citadas

1. Reto_IA_-_Deuna_-_I2H2026.pdf
2. Estudio – Radiografía de las tiendas ed barrio en Ecuador 2025 ..., fecha de acceso: abril 16, 2026, https://www.ret.ec/investigacion/
3. Llama 3 vs. Mistral vs. Phi-3: Which Self-Hosted LLM Should You Choose for Business Tasks? - ForgeNEX, fecha de acceso: abril 16, 2026, https://www.forgenex.com/en/blog/llama-3-vs-mistral-vs-phi-3-que-llm-autohospedado-elegir-para-tareas-de-negocio
4. Both Ends Count! Just How Good are LLM Agents at Text-to-“Big SQL”? - arXiv, fecha de acceso: abril 16, 2026, https://arxiv.org/html/2602.21480v4
5. Which LLM writes the best analytical SQL? - Tinybird, fecha de acceso: abril 16, 2026, https://www.tinybird.co/blog/which-llm-writes-the-best-sql
6. The Text-to-SQL Performance Cliff (2026): Why “Natural Language ..., fecha de acceso: abril 16, 2026, https://medium.com/@visrow/the-text-to-sql-performance-cliff-2026-why-natural-language-to-sql-breaks-a7281a23dbea
7. Natural Language Queries on Pandas with LangChain and DuckDB - Adam Cowley, fecha de acceso: abril 16, 2026, https://adamcowley.co.uk/posts/llm-sql-duckdb/
8. PySpark vs Pandas vs DuckDB: Which Framework Actually Fits Your Data Scale? - Medium, fecha de acceso: abril 16, 2026,

https://medium.com/@renan.de.moraes777/pyspark-vs-pandas-vs-duckdb-which-framework-actually-fits-your-data-scale-89e6c189a86b
9. DuckDB vs. Polars vs. Pandas: Benchmark & Comparison - codecentric AG, fecha de acceso: abril 16, 2026, https://www.codecentric.de/en/knowledge-hub/blog/duckdb-vs-dataframe-libraries
10. Build a LangChain SQL Agent with DuckDB and MotherDuck, fecha de acceso: abril 16, 2026, https://motherduck.com/blog/langchain-sql-agent-duckdb-motherduck/
11. A Lightweight Local SQL Agent Using LLMs and DuckDB for Business Analytics - TechRxiv, fecha de acceso: abril 16, 2026, https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.175259650.02426352
12. We Benchmarked DuckDB, SQLite, and Pandas on 1M Rows: Here's What Happened, fecha de acceso: abril 16, 2026, https://www.kdnuggets.com/we-benchmarked-duckdb-sqlite-and-pandas-on-1m-rows-heres-what-happened
13. Chat with CSV & Excel Files Using Local AI, fecha de acceso: abril 16, 2026, https://localaimaster.com/blog/local-ai-data-analyst
14. LangGraph vs CrewAI vs AutoGen: Which AI Agent Framework Should Your Enterprise Use in 2026? - Towards AI, fecha de acceso: abril 16, 2026, https://pub.towardsai.net/langgraph-vs-crewai-vs-autogen-which-ai-agent-framework-should-your-enterprise-use-in-2026-3a9ebb407b09
15. Comparing AI agent frameworks: CrewAI, LangGraph, and BeeAI - IBM Developer, fecha de acceso: abril 16, 2026, https://developer.ibm.com/articles/awb-comparing-ai-agent-frameworks-crewai-langgraph-and-beeai/
16. Crewai vs LangGraph: Know The Differences - TrueFoundry, fecha de acceso: abril 16, 2026, https://www.truefoundry.com/blog/crewai-vs-langgraph
17. I Tried 20+ AI Frameworks: Here are My Top 10 Recommendations for 2026 | by Soma | Javarevisited | Mar, 2026, fecha de acceso: abril 16, 2026, https://medium.com/javarevisited/i-tried-20-ai-frameworks-here-are-my-top-10-recommendations-for-2026-927168fed61c
18. Tutorial: How to build a simple text-to-SQL agent that can automatically recover from bad SQL : r/DuckDB - Reddit, fecha de acceso: abril 16, 2026, https://www.reddit.com/r/DuckDB/comments/1sgqlcd/tutorial_how_to_build_a_simple_texttosql_agent/
19. Text-to-SQL: Comparison of LLM Accuracy - AIMultiple, fecha de acceso: abril 16, 2026, https://aimultiple.com/text-to-sql
20. 10 Best Open-Source LLM Models (2025 Updated): Llama 4, Qwen 3 and DeepSeek R1, fecha de acceso: abril 16, 2026, https://huggingface.co/blog/daya-shankar/open-source-llms
21. Top 8 Open-Source LLMs to Watch in 2025 - JetRuby Agency, fecha de acceso: abril 16, 2026, https://jetruby.com/blog/top-8-open-source-llms-to-watch-in-2025/
22. The 11 best open-source LLMs for 2025 - n8n Blog, fecha de acceso: abril 16,

2026, https://blog.n8n.io/open-source-llm/
23. How to Choose the Right GPU for vLLM Inference - DigitalOcean, fecha de acceso: abril 16, 2026, https://www.digitalocean.com/community/conceptual-articles/vllm-gpu-sizing-configuration-guide
24. Should I run Llama 70B on an NVIDIA H100 or A100? | AI FAQ | Jarvis Labs, fecha de acceso: abril 16, 2026, https://jarvislabs.ai/ai-faqs/should-i-run-llama-70b-on-an-nvidia-h100-or-a100
25. Optimizing vLLM Performance on A100 80GB: GPU Benchmark Insights - Database Mart, fecha de acceso: abril 16, 2026, https://www.databasemart.com/blog/vllm-gpu-benchmark-a100-80gb
26. vLLM vs Triton vs TGI: Choosing the Right LLM Serving Framework - Clarifai, fecha de acceso: abril 16, 2026, https://www.clarifai.com/blog/model-serving-framework/
27. 2025-01 vLLM/Llama 3.3 70B FP8 tokens/joule - GitHub Gist, fecha de acceso: abril 16, 2026, https://gist.github.com/lhl/bf81a9c7dfc4244c974335e1605dcf22
28. vLLM or llama.cpp: Choosing the right LLM inference engine for your use case, fecha de acceso: abril 16, 2026, https://developers.redhat.com/articles/2025/09/30/vllm-or-llamacpp-choosing-right-llm-inference-engine-your-use-case
29. LLM Compressor is here: Faster inference with vLLM | Red Hat Developer, fecha de acceso: abril 16, 2026, https://developers.redhat.com/articles/2024/08/14/llm-compressor-here-faster-inference-vllm
30. Any benchmarks for quantized LLM against the full precision models? - Reddit, fecha de acceso: abril 16, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1ifqfpb/any_benchmarks_for_quantized_llm_against_the_full/
31. Comparative Analysis of Large Language Model Inference Serving Systems: A Performance Study of vLLM and HuggingFace TGI - arXiv, fecha de acceso: abril 16, 2026, https://arxiv.org/html/2511.17593v1
32. Llama 3.3 70B very slow - vLLM Forums, fecha de acceso: abril 16, 2026, https://discuss.vllm.ai/t/llama-3-3-70b-very-slow/1715
33. [Usage]: Extremely slow inference with Llama 3.1 70b Instruct · Issue #7567 - GitHub, fecha de acceso: abril 16, 2026, https://github.com/vllm-project/vllm/issues/7567
34. Running GPT-OSS with vLLM on Supercomputers (SLURM + Singularity) - Medium, fecha de acceso: abril 16, 2026, https://medium.com/@qualis2006/running-gpt-oss-with-vllm-on-supercomputers-slurm-singularity-a6ab70f7e044
35. Distributed vLLM Inference - Bristol Centre for Supercomputing Documentation, fecha de acceso: abril 16, 2026, https://docs.isambard.ac.uk/user-documentation/tutorials/distributed-inference/
36. Best practice for running multi-node vLLM inference on Slurm (port conflicts, orchestration), fecha de acceso: abril 16, 2026,

https://www.reddit.com/r/SLURM/comments/1qoe0ng/best_practice_for_running_multinode_vllm/
37. Running vLLM multi-node data parallel with SLURM - RL Integration, fecha de acceso: abril 16, 2026, https://discuss.vllm.ai/t/running-vllm-multi-node-data-parallel-with-slurm/1362
38. Chat with your CSV: Visualize Your Data with Langchain and ..., fecha de acceso: abril 16, 2026, https://dev.to/ngonidzashe/chat-with-your-csv-visualize-your-data-with-langchain-and-streamlit-ej7
39. Chainlit vs. Streamlit Comparison - SourceForge, fecha de acceso: abril 16, 2026, https://sourceforge.net/software/compare/Chainlit-vs-Streamlit/
40. Streamlit vs Chainlit: Which is Better for AI Apps? | Beginners Guide - YouTube, fecha de acceso: abril 16, 2026, https://www.youtube.com/watch?v=GqltEDPixX0
41. Overview - Chainlit, fecha de acceso: abril 16, 2026, https://docs.chainlit.io/get-started/overview
42. Chainlit: A Guide With Practical Examples - DataCamp, fecha de acceso: abril 16, 2026, https://www.datacamp.com/tutorial/chainlit
43. OpenAI - Chainlit, fecha de acceso: abril 16, 2026, https://docs.chainlit.io/integrations/openai
44. Plotly - Chainlit, fecha de acceso: abril 16, 2026, https://docs.chainlit.io/api-reference/elements/plotly
45. Chainlit #13: Plotly in Chatbot Draw Bar, Pie & Scatter Charts - YouTube, fecha de acceso: abril 16, 2026, https://www.youtube.com/watch?v=ljnOONAW6T0
46. Plotly elements not rendered on resume with custom SQLAlchemy data layer in Chainlit #2315 - GitHub, fecha de acceso: abril 16, 2026, https://github.com/Chainlit/chainlit/issues/2315
47. Agentic Patterns and Implementation with Agentforce - Salesforce Architects, fecha de acceso: abril 16, 2026, https://architect.salesforce.com/docs/architect/fundamentals/guide/agentic-patterns
48. AI Agents for Business Intelligence: Guide + Use Cases - Domo, fecha de acceso: abril 16, 2026, https://www.domo.com/learn/article/ai-agents-in-business-intelligence
49. 15 Ways to Use AI Agents for Data Analysis - MindStudio, fecha de acceso: abril 16, 2026, https://www.mindstudio.ai/blog/ai-agents-data-analysis-ways
50. on_chat_start function called if background task takes too long · Issue #113 - GitHub, fecha de acceso: abril 16, 2026, https://github.com/Chainlit/chainlit/issues/113
51. Async / Sync - Chainlit, fecha de acceso: abril 16, 2026, https://docs.chainlit.io/guides/sync-async
52. Background Tasks - FastAPI, fecha de acceso: abril 16, 2026, https://fastapi.tiangolo.com/tutorial/background-tasks/
53. Message - Chainlit, fecha de acceso: abril 16, 2026, https://docs.chainlit.io/concepts/message
54. Message - Chainlit, fecha de acceso: abril 16, 2026,

https://docs.chainlit.io/api-reference/message
55. The Missing Layer in AI Apps: Why Your LLM Product Needs Chat ..., fecha de acceso: abril 16, 2026, https://trtc.io/blog/details/ai-app-chat-layer-llm-push-notifications
56. Teradata AI Agent for Data Analysis: Autonomous AI for Smarter Insights, fecha de acceso: abril 16, 2026, https://www.teradata.com/insights/ai-and-machine-learning/teradata-ai-agent-for-data-analysis
57. In Spanish - The Best Prompt Structures - Basic to Intermediate Part I - YouTube, fecha de acceso: abril 16, 2026, https://www.youtube.com/watch?v=257yyJkaGeE
58. 40+ Best AI Prompts for Small Businesses - INSIDEA, fecha de acceso: abril 16, 2026, https://insidea.com/blog/ai/prompts/best-ai-prompts-for-small-businesses/
59. AI prompts for small-business owners and entrepreneurs | Google Workspace, fecha de acceso: abril 16, 2026, https://workspace.google.com/intl/en_za/resources/ai/prompts-for-small-business/
60. 125 palabras y expresiones ecuatorianas (con su significado) - Psicología y Mente, fecha de acceso: abril 16, 2026, https://psicologiaymente.com/reflexiones/palabras-expresiones-ecuatorianas
61. Diccionario ecuatoriano: Palabras, vocabulario y frases que usamos en Ecuador, fecha de acceso: abril 16, 2026, https://www.chokotrip.info/diccionario-ecuatoriano/
62. 15 palabras y expresiones del argot ecuatoriano que debes conocer - Inhispania, fecha de acceso: abril 16, 2026, https://inhispania.com/es/15-palabras-y-expresiones-ecuatorianas/
63. I built a multilingual AI Marketing Prompt System (English/Spanish/Ukrainian) - feedback welcome : r/PromptEngineering - Reddit, fecha de acceso: abril 16, 2026, https://www.reddit.com/r/PromptEngineering/comments/1ov3fyd/i_built_a_multilingual_ai_marketing_prompt_system/
64. Harnessing RAG: LangChain vs. Pandas for Superior Conversational Analytics - Encora, fecha de acceso: abril 16, 2026, https://www.encora.com/interface/harnessing-rag-langchain-vs.-pandas-for-superior-conversational-analytics
65. How DuckDB Complements Pandas for Large-Scale Analytics - DigitalOcean, fecha de acceso: abril 16, 2026, https://www.digitalocean.com/community/tutorials/duckdb-complements-pandas-for-large-scale-analytics